# Taller CRUD

## Spring Boot – Thymeleaf – PostgreSQL

Mejoras en la Aplicación Web de Gestión de Individuos


**Juan Manuel Lagos**

Ficha: **3114733**

Análisis y Desarrollo de Software (ADSO)

Instructora: **Diana Delgado**

# Tabla de Contenido


**1 Introducción** . . . . . **2**

**2 Objetivo** . . . . . **2**

**3 Herramientas que Usé** . . . . . **2**

**4 Qué Hace Cada Capa del Proyecto** . . . . . **2**

4.1 Entity – Individuo.java . . . . . 3

4.2 DAO – IndividuoDao.java . . . . . 3

4.3 Service – IIndividuoService.java e IndividuoService.java . . . . . 4

4.4 Controller – HomeController.java . . . . . 5

4.5 View – Plantillas HTML con Thymeleaf . . . . . 6

**5 Mejoras que Implementé** . . . . . **7**

**6 Pruebas del CRUD** . . . . . **8**

6.1 CREATE – Crear un Registro . . . . . 8

6.2 UPDATE – Editar un Registro . . . . . 9

6.3 DELETE – Eliminar un Registro . . . . . 10

6.4 Verificación en la Base de Datos . . . . . 10

**7 Estructura del Proyecto** . . . . . **11**

**8 Conclusiones** . . . . . **12**

**9 Referencias** . . . . . **13**

# 1. Introducción

Este documento es la solución al taller que nos puso la instructora Diana Delgado sobre Spring Boot CRUD. Lo que teníamos que hacer era tomar el proyecto que ya veníamos trabajando en clase, que es una aplicación web para gestionar individuos hecha con Spring Boot, Thymeleaf y PostgreSQL, y hacerle unas mejoras específicas.

Las mejoras que me tocaba hacer eran: agregar la columna de **teléfono** a la tabla, **separar** las acciones de Ver, Editar y Eliminar, poner un **botón de "Nuevo Individuo"**, y comprobar que se pueda **crear, editar y eliminar** desde la interfaz verificando que se refleje en la base de datos.

También tenía que explicar con mis propias palabras qué hace cada capa del proyecto.

# 2. Objetivo

Mejorar la aplicación CRUD que hicimos en clase con Spring Boot, Thymeleaf y PostgreSQL, agregándole las funcionalidades que pide el taller y demostrando con capturas que todo funciona.

# 3. Herramientas que Usé

> **Java 17** – lenguaje del proyecto | **Spring Boot** – framework web | **Thymeleaf** – motor de plantillas | **PostgreSQL** – base de datos | **Visual Studio Code** – editor de código | **Maven** – dependencias | **Google Chrome** – pruebas

# 4. Qué Hace Cada Capa del Proyecto

El proyecto usa la arquitectura MVC (Modelo–Vista–Controlador). Cada capa tiene un trabajo específico y así el código queda organizado. A continuación explico cada una.

## 4.1 Entity – Individuo.java

Esta es la clase que representa a una persona en el sistema. Tiene los datos como nombre, apellido, edad, correo y teléfono. Cada atributo se convierte en una columna de la tabla en la base de datos gracias a la anotación `@Entity`. Yo lo veo como el "molde" de los datos: le dice a la app qué información va a guardar de cada individuo. También tiene `@Id` para que cada registro tenga su propio número único que se genera solo.

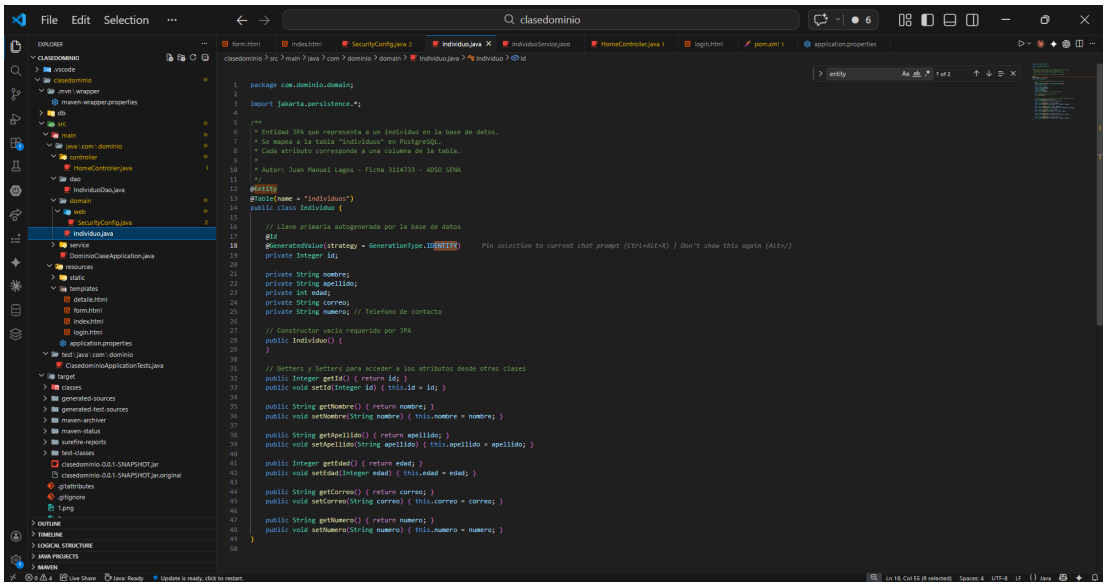


Figura 1: Código de Individuo.java – la entidad que representa la tabla en la base de datos.

## 4.2 DAO – IndividuoDao.java

El DAO es lo que conecta la aplicación con la base de datos. Es una interfaz que extiende de `CrudRepository`, y lo bueno es que Spring ya trae listos los métodos para guardar, buscar, listar y borrar sin tener que escribir SQL. También le agregué un método para buscar por nombre.

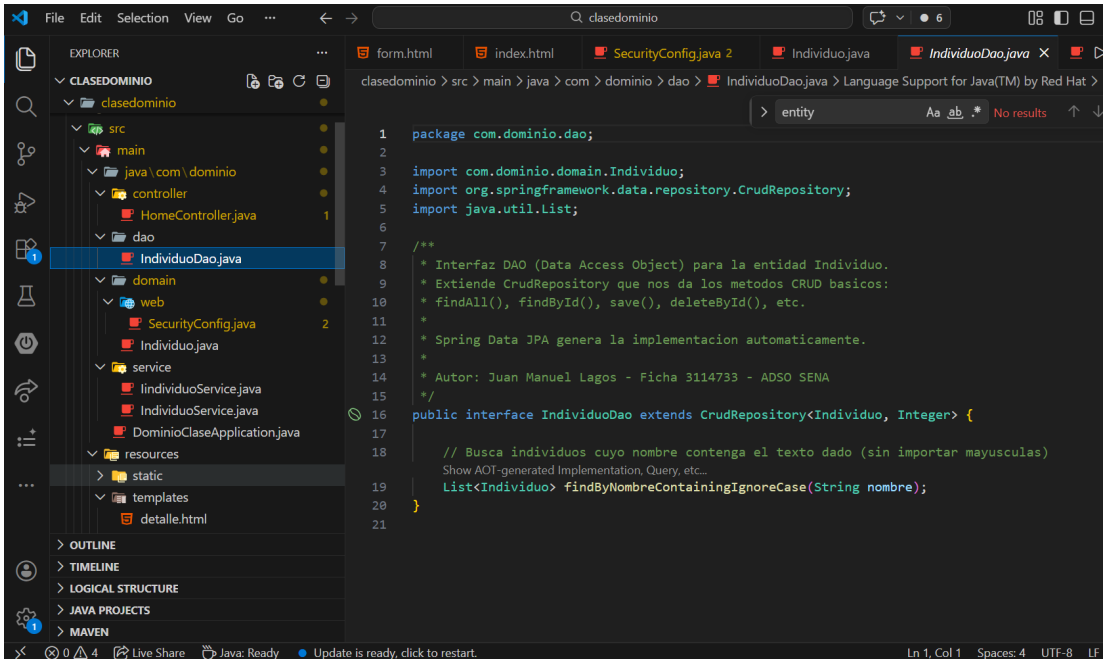


Figura 2: Código de IndividuoDao.java – interfaz que conecta con PostgreSQL.

## 4.3 Service – IIndividuoService.java e IndividuoService.java

El servicio está en medio del controlador y el DAO. Hice una interfaz (`IIndividuoService`) donde puse los métodos que necesitaba: listar, buscar, guardar y eliminar. Después la clase `IndividuoService` implementa esos métodos llamando al DAO. La idea de usar interfaz es por si toca cambiar algo después, solo se cambia la implementación y ya.

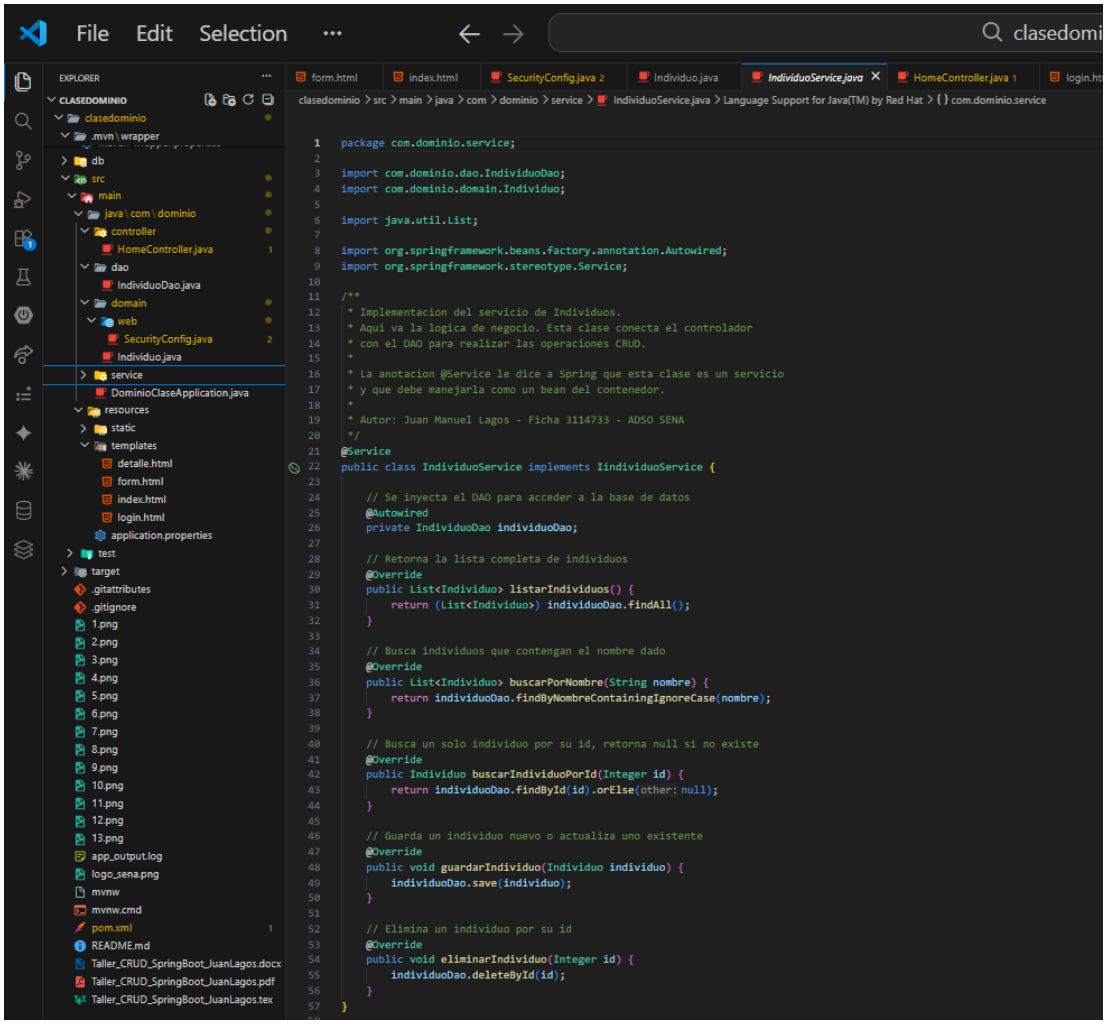


Figura 3: Código de la interfaz IIndividuoService.java y su implementación IndividuoService.java.

## 4.4 Controller – HomeController.java

El controlador es como el "cerebro" de la app: recibe lo que el usuario hace en el navegador y decide qué mostrar. Si entro a `localhost:8080/` me muestra la lista, si voy a `/nuevo` me abre el formulario, y así con cada ruta. También tiene las rutas de `/guardar`, `/editar`, `/eliminar` y `/detalle`.

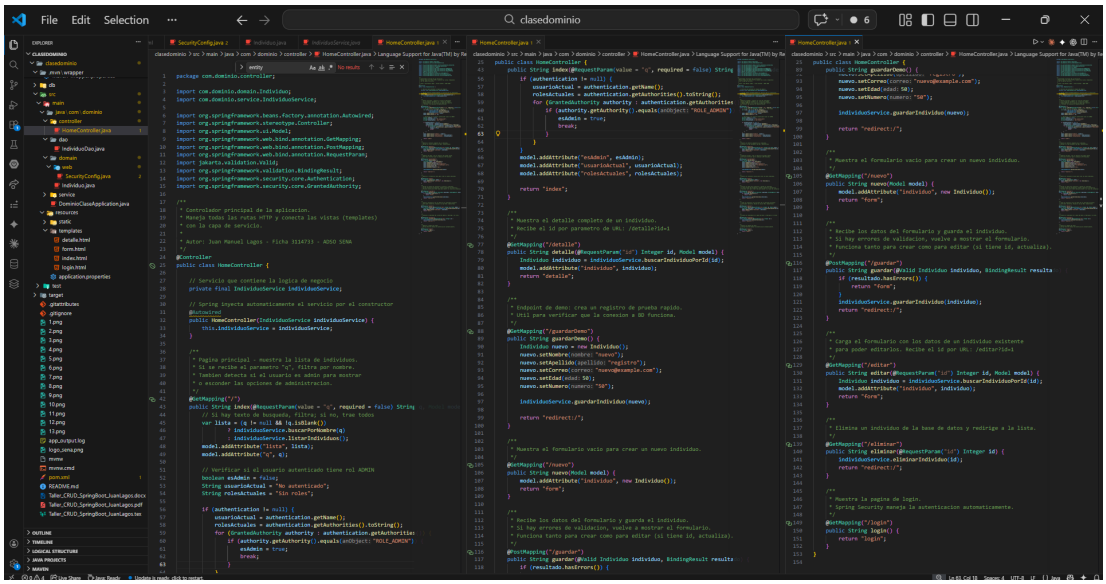

Figura 4: Código de HomeController.java – el controlador con las rutas del CRUD.

## 4.5 View – Plantillas HTML con Thymeleaf

Las vistas son las páginas que uno ve en el navegador. Usé Thymeleaf para que los datos de la base de datos se muestren en el HTML. Tengo tres: `index.html` que es la lista, `form.html` que es el formulario y `detalle.html` que muestra toda la info de una persona.

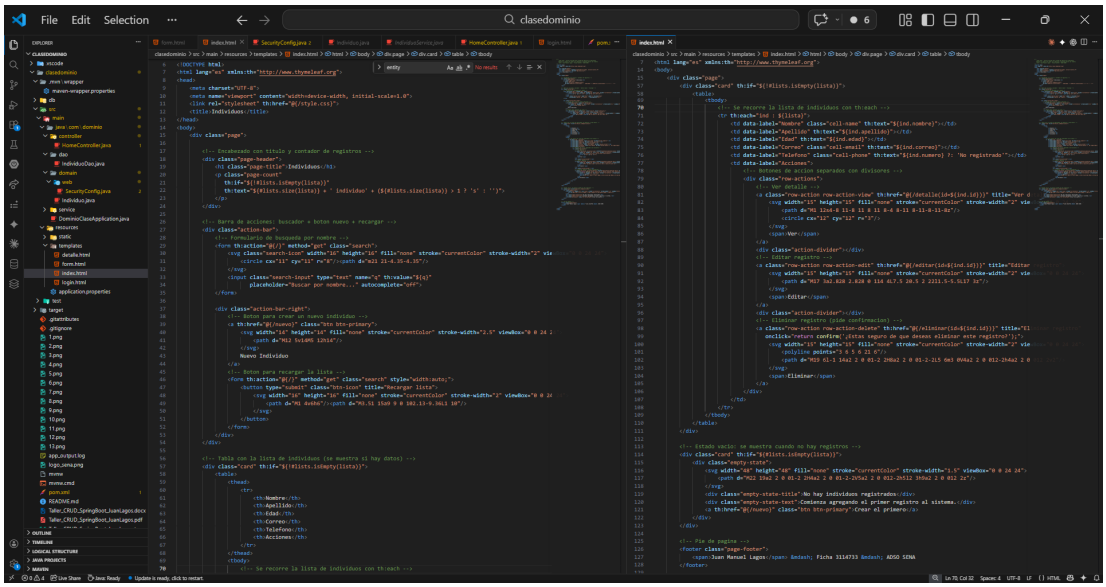

Figura 5: Código de index.html – la vista principal con la tabla de individuos.

# 5. Mejoras que Implementé

El taller pedía tres mejoras específicas. En la siguiente captura se pueden ver las tres implementadas en la página principal:

- La columna de **Teléfono** ya aparece en la tabla junto a los demás datos.
- Las acciones **Ver**, **Editar** y **Eliminar** están separadas visualmente con delimitadores.
- El botón **"Nuevo Individuo"** está visible en la parte superior de la tabla.

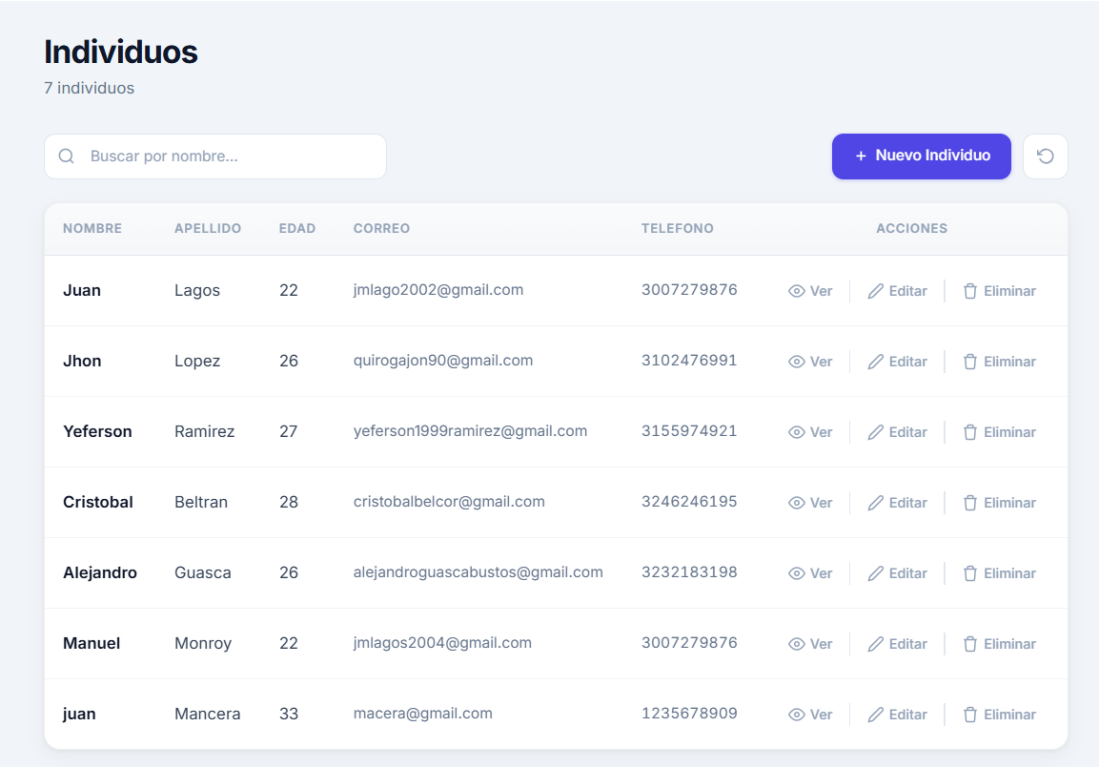


Figura 6: Página principal mostrando las 3 mejoras: columna teléfono, acciones separadas y botón Nuevo Individuo.

Para el teléfono me aseguré de que la entidad tuviera el campo `numero`, le agregué la columna a la tabla del `index.html` y también puse el campo en el formulario. Las acciones las separé con barras para que se vean claras y la de eliminar pregunta antes de borrar. El botón de "Nuevo Individuo" va a la ruta `/nuevo`.

# 6. Pruebas del CRUD

Aquí muestro que cada operación funciona desde la interfaz web y se refleja en la base de datos.

## 6.1 CREATE – Crear un Registro

Le di clic a “Nuevo Individuo”, llené el formulario con datos de prueba y guardé. El individuo nuevo apareció en la lista inmediatamente.

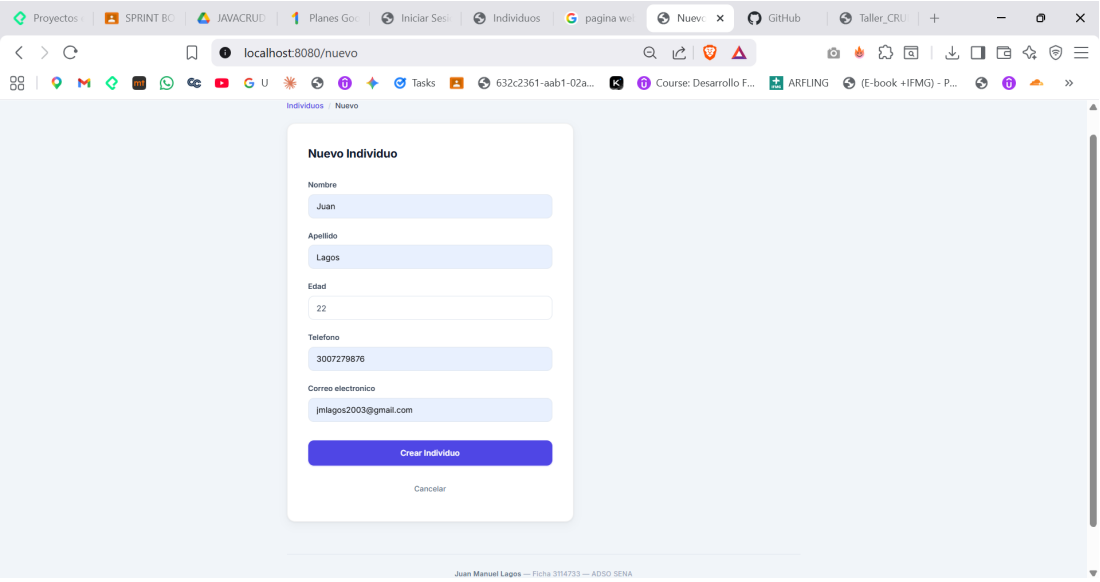


Figura 7: Formulario llenado con los datos del nuevo individuo.

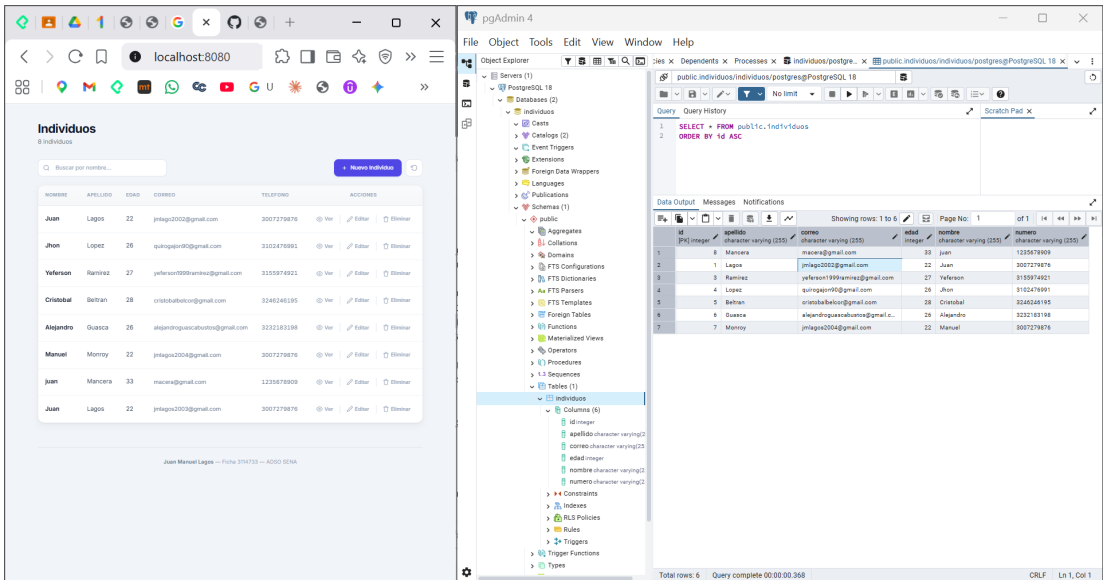


Figura 8: Lista mostrando que el registro nuevo ya aparece creado.

## 6.2 UPDATE – Editar un Registro

Seleccioné un registro y le di "Editar". El formulario se abrió con los datos ya cargados. Modifiqué lo que necesitaba, guardé y al volver a la lista los cambios ya se veían.

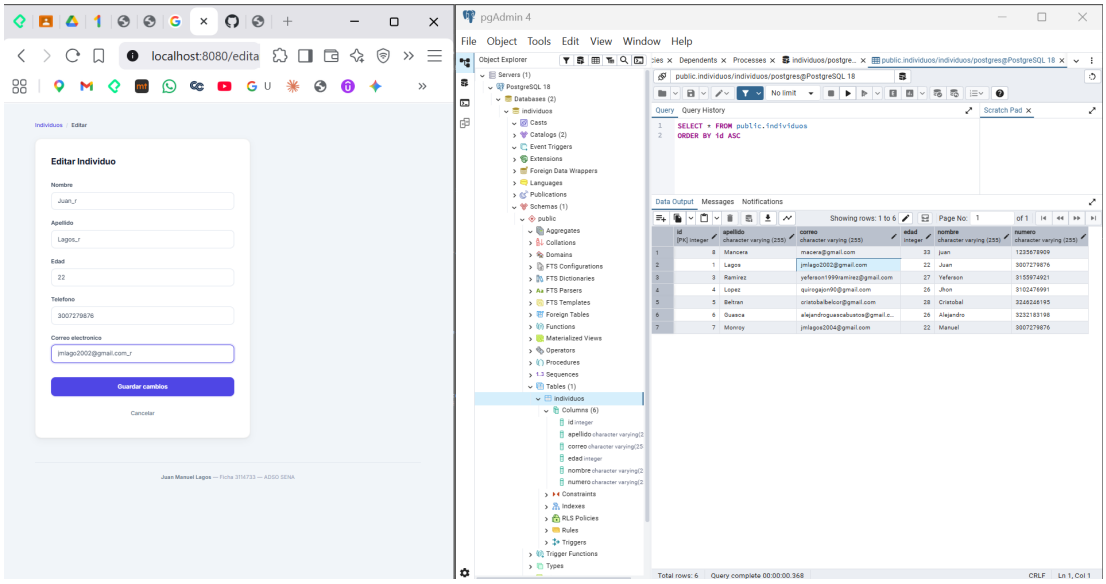


Figura 9: Formulario de edición con los datos cargados y modificados.

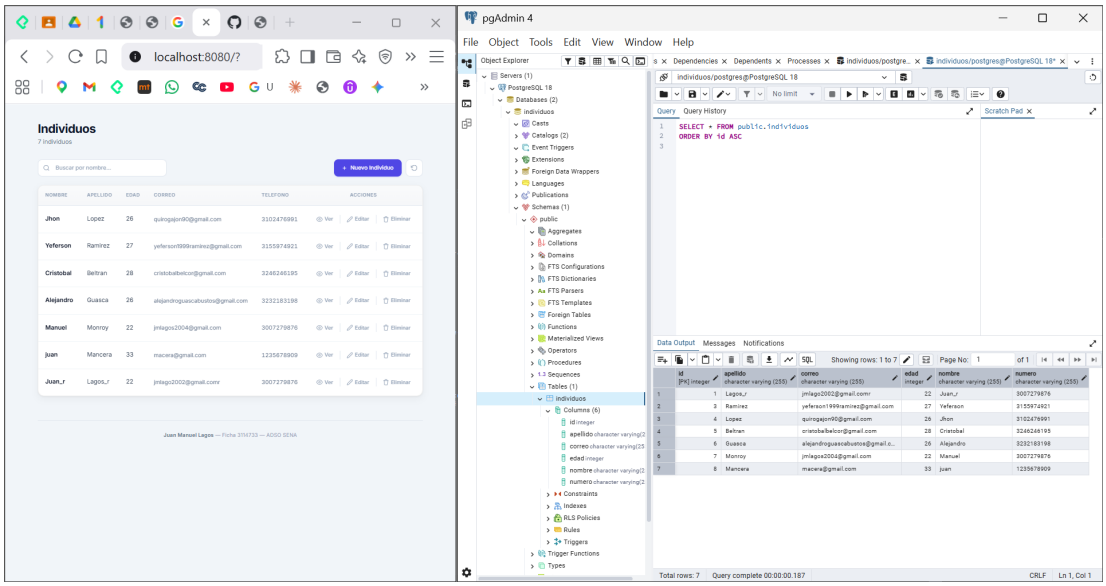


Figura 10: Lista mostrando los datos actualizados después de guardar.

## 6.3 DELETE – Eliminar un Registro

Le di clic a "Eliminar" en un registro. Me salió la confirmación, acepté y el registro desapareció de la lista.

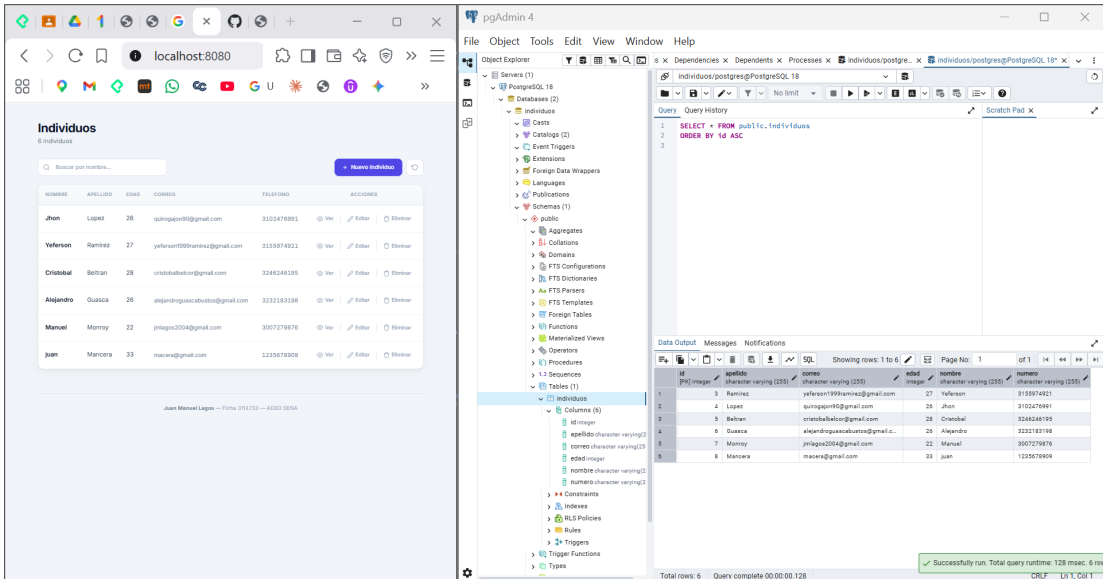


Figura 11: Lista después de eliminar – el registro ya no está.

## 6.4 Verificación en la Base de Datos

Para estar seguro de que los cambios no eran solo en la página sino que de verdad quedaban guardados, abrí PostgreSQL e hice una consulta. Ahí se ve que los registros que creé, edité y borré sí se reflejaron en la tabla.

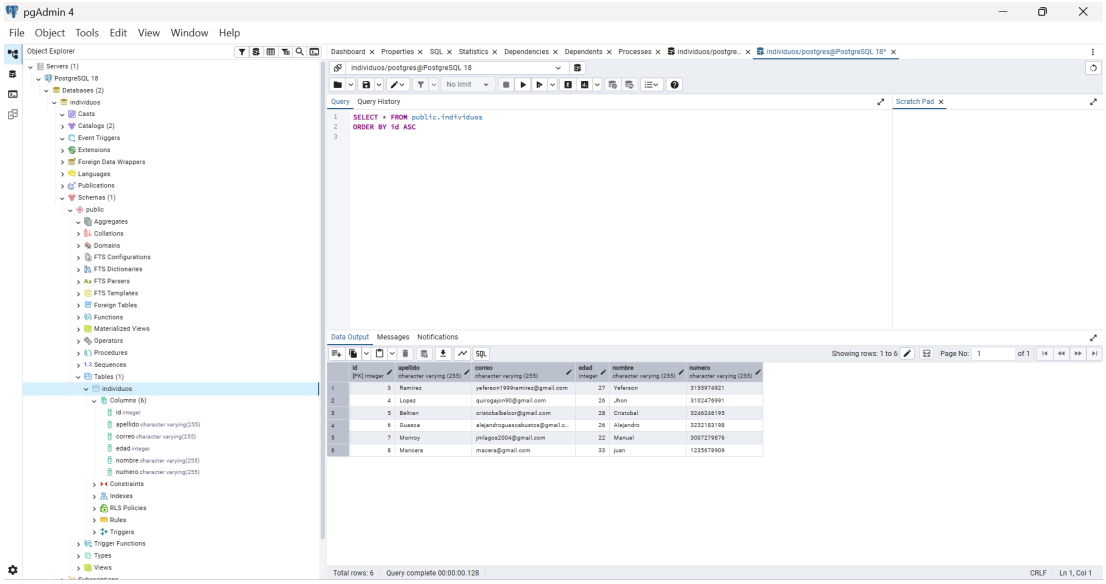


Figura 12: Consulta a la base de datos PostgreSQL mostrando los registros actualizados.

# 7. Estructura del Proyecto

Así están organizadas las carpetas de mi proyecto. Cada capa tiene su carpeta dentro de `src/main/java/com/dominio/` y las vistas están en `resources/templates/`.

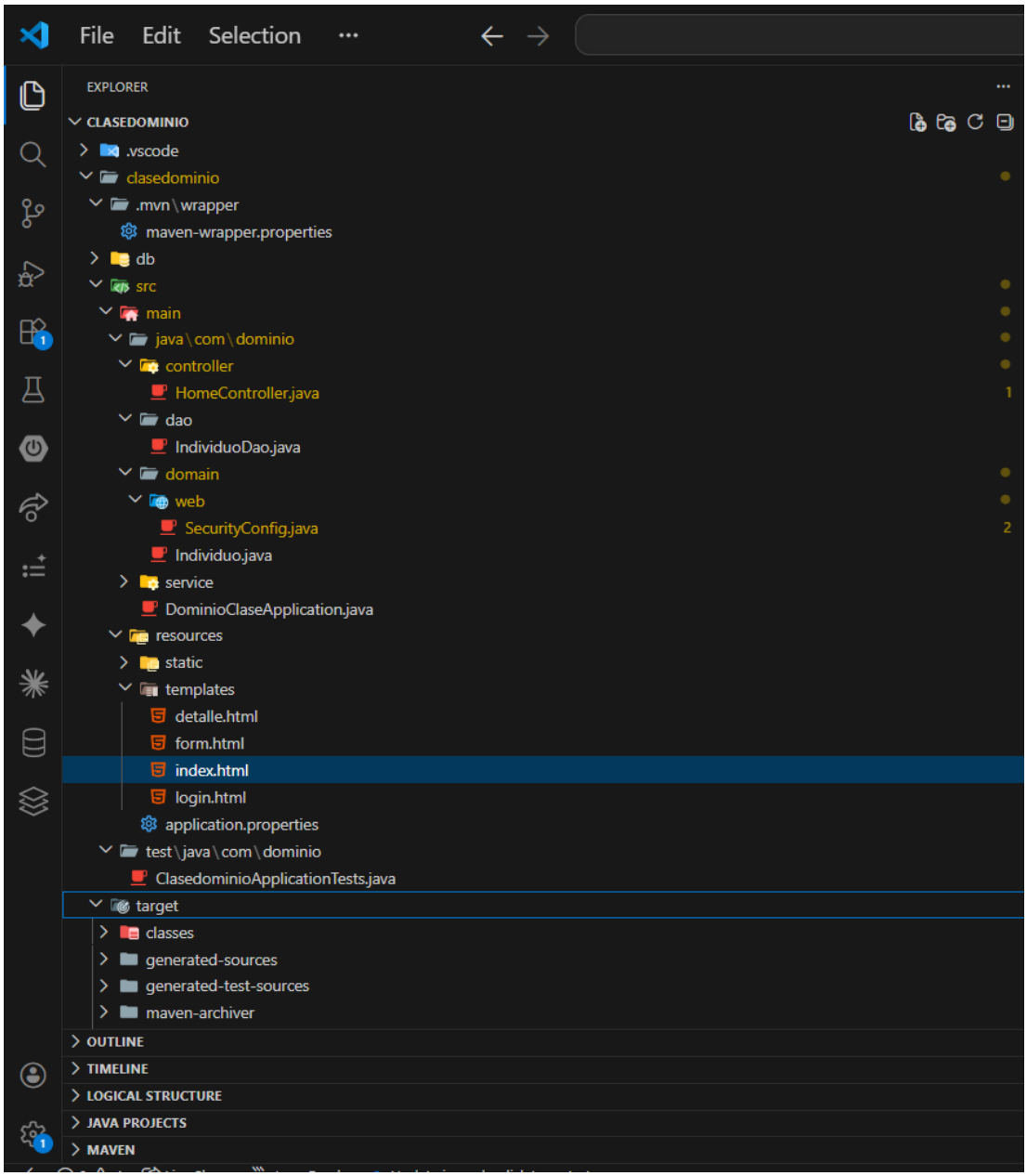


Figura 13: Estructura de carpetas del proyecto en el explorador de Visual Studio Code.

# 8. Conclusiones

Con este taller practicé lo que vimos en clase sobre Spring Boot y la arquitectura MVC. Me quedó más claro cómo se conectan las capas entre sí: la Entity con la base de datos, el DAO que hace las consultas, el Service con la lógica, el Controller que maneja las rutas y las Vistas que muestran todo.

Las mejoras que hice dejaron la app más completa. Ya se ve el teléfono, las acciones se distinguen bien y el botón de crear funciona. Lo más importante es que comprobé que todo lo que hago en la página web queda guardado en PostgreSQL.

## 9. Referencias

Delgado, D. (2026). *Guía Sprint Boot CRUD* [Material de clase]. SENA.

Spring. (s.f.). *Spring Boot Reference Documentation*. https://docs.spring.io/spring-boot/

Thymeleaf. (s.f.). *Tutorial: Using Thymeleaf*. https://www.thymeleaf.org/documentation.html